

UNIVERSITAS GADJAH MADA

Departemen Ilmu Komputer dan Elektronika

Tutorial GPU: Pemrograman CUDA C++

Delapan Exercise dan Mini-Project Stencil 2D: Inspeksi Device, Kernel, Vector Add, Timing, Grid-Stride, Reduksi, Matmul Tiled, dan Compute Sanitizer

Nama: Aufa Akmal Bunaya

NIM: 23/515767/PA/22027

Repository GitHub: github.com/aufaakmalbunaya/cuda-tutorial

Daftar Isi

1	Pendahuluan	2
1.1	Lingkungan dan persiapan	2
2	Exercise 1: Inspeksi GPU yang tersedia	2
3	Exercise 2: Kernel pertama	2
4	Exercise 3: Penjumlahan vektor dengan memori device eksplisit	3
5	Exercise 4: Waktu kernel vs waktu termasuk transfer	3
6	Exercise 5: Kernel grid-stride, tuning launch, bandwidth efektif	4
7	Exercise 6: Reduksi dengan shared memory	5
8	Exercise 7: Perkalian matriks naive vs tiled	6
9	Exercise 8: Diagnosis error dengan Compute Sanitizer	7
10	Mini-project: Stencil 2D lima titik	7
11	Refleksi: perlambatan, perbaikan, dan bug	7
12	Reproduksi	8
13	Kesimpulan	8

Pendahuluan

Tugas ini mengerjakan tutorial pemrograman GPU dengan CUDA C++: delapan exercise berurutan yang mencakup inspeksi device, peluncuran kernel pertama, penambahan vektor dengan manajemen memori device eksplisit, pemisahan waktu kernel vs waktu transfer, kernel grid-stride dengan tuning konfigurasi launch dan bandwidth efektif, reduksi memakai shared memory, perbandingan perkalian matriks naive vs tiled, serta diagnosis error dengan Compute Sanitizer, ditutup dengan mini-project stencil 2D lima titik. Seluruh program dikompilasi dan dijalankan secara nyata pada GPU NVIDIA; semua angka pada laporan ini adalah hasil pengukuran nyata (bukan estimasi), diringkas sebagai median dari beberapa run.

Lingkungan dan persiapan

VM lokal yang tersedia tidak memiliki GPU (`nvidia-smi` tidak ada, `nvcc` tidak ada, tidak ada device `/dev/nvidia*`), sehingga seluruh pekerjaan dijalankan pada notebook Kaggle dengan akselerator GPU. Spesifikasi lingkungan pada Tabel 1 dan toolchain pada Tabel 2.

Tabel 1: Spesifikasi GPU (hasil `nvidia-smi` dan program inspeksi).

Komponen	Nilai
GPU	Tesla T4, 15360 MiB (15 GiB)
Compute capability	7.5
Streaming multiprocessors	40
Warp size	32 thread
Maksimum thread per block	1024
Global memory	14.56 GiB (terpakai)
Shared memory per block	49152 byte (48 KiB)
Driver	580.178.04

Tabel 2: Toolchain kompilasi.

Komponen	Nilai
CUDA toolkit	12.8 (V12.8.93)
Flag kompilasi	<code>-std=c++17 -O3 -lineinfo -arch=native</code>
Helper makro	<code>CUDA_CHECK</code> pada <code>cuda_check.cuh</code> (cek tiap API call)
Platform eksekusi	Notebook Kaggle, akselerator GPU

Exercise 1: Inspeksi GPU yang tersedia

Berkas: `src/device_info.cu`. Program memanggil `cudaGetDeviceProperties` dan mencetak properti device 0. Hasilnya sesuai Tabel 1: Tesla T4 dengan compute capability 7.5, 40 SM, dan memori global 14.56 GiB. Flag `-arch=native` pada `nvcc` 12.8 berhasil dipakai langsung pada T4 tanpa perlu menentukan `sm_75` secara manual.

Exercise 2: Kernel pertama

Berkas: `src/hello_gpu.cu`. Kernel `hello` menghitung indeks global `i = blockIdx.x * blockDim.x + threadIdx.x` dan mencetak `block`, `local` (`threadIdx.x`), dan `global` hanya bila `i < n`. Dengan

`n=20` dan 8 thread per block, dibutuhkan 3 block sehingga 24 thread diluncurkan dan 20 di antaranya aktif; guard `i < n` mencegah 4 thread sisa menulis/mencetak di luar rentang.

Satu detail menarik dari output nyata: baris `block=2` tercetak *sebelum* `block=0` — urutan `printf` antar block tidak deterministik karena block dieksekusi sejajar oleh SM yang berbeda. Host mencetak konfirmasi setelah `cudaDeviceSynchronize`.

Varian modifikasi (`src/hello_gpu_17.cu`, `n=17`, 16 thread per block): tepat 2 block diluncurkan (32 thread), 17 aktif, sesuai prediksi `ceil(17/16) = 2`.

Exercise 3: Penjumlahan vektor dengan memori device eksplisit

Berkas: `src/vector_add.cu`. Alur program mengikuti tutorial: `cudaMalloc` untuk tiga vektor, `cudaMemcpy` host-ke-device, launch kernel `vec_add` dengan 256 thread per block, `cudaMemcpy` device-ke-host, verifikasi terhadap hasil CPU, lalu `cudaFree`. Untuk `n=1000003` dipakai 3907 block, `errors=0`, dengan `first=0.00` dan `last=9.25` sesuai inisialisasi `a[i]=i*0.25f`, `b[i]=i*0.5f` (yakni `c[i]=0.75*i`).

Uji ukuran tepi (rekompilasi per ukuran, sesuai permintaan tutorial) semuanya lolos:

n	block	thread	errors
1	1	256	0
255	1	256	0
256	1	256	0
257	2	256	0
1000003	3907	256	0

Kasus `n=257` butuh 2 block; thread sisa pada block kedua ditangani guard batas dengan benar.

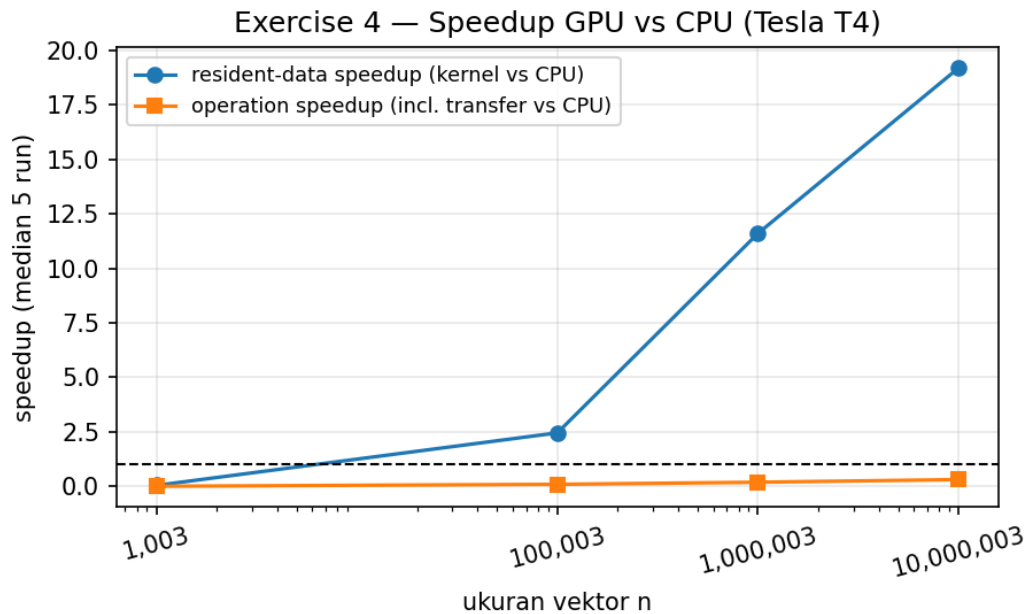
Modifikasi (`src/vector_add_alpha.cu`): kernel diubah menjadi `c[i] = 2*a[i] + b[i]`; untuk `n=1000003` hasilnya `errors=0`.

Exercise 4: Waktu kernel vs waktu termasuk transfer

Berkas: `src/vector_benchmark.cu`. Program mengukur tiga hal dengan CUDA event: waktu kernel saja (`kernel`, hanya launch + eksekusi), waktu operasi penuh (`operation`: alokasi, dua H2D copy, kernel, satu D2H copy), dan waktu CPU sebagai pembanding. Dilaporkan dua speedup: *resident-data speedup* = CPU/kernel (data sudah di device) dan *operation speedup* = CPU/operation (termasuk transfer). Setiap ukuran dijalankan 5 kali; Tabel 3 memakai median.

Tabel 3: Exercise 4 — median 5 run per ukuran (Tesla T4).

n	CPU (ms)	kernel (ms)	operasi (ms)	speedup residu	speedup operasi
1003	0.0004	0.0082	0.0443	0.05	0.01
100003	0.0344	0.0162	0.4099	2.13	0.08
1000003	0.5801	0.0525	3.0029	11.04	0.19
10000003	8.8437	0.4576	25.9355	19.33	0.34



Gambar 1: Speedup GPU vs CPU terhadap ukuran vektor (median 5 run). Speedup kernel-only tumbuh hingga $\approx 19.3\times$ pada $n = 10^7$, tetapi speedup termasuk transfer tidak pernah melewati 1.

Dua batas waktu (*timing boundaries*) didefinisikan eksplisit: **kernel** hanya melingkupi launch kernel (diukur dengan CUDA event, tanpa transfer), sedangkan **operation** mencakup alokasi device, copy H2D/D2H, dan kernel. Hasilnya menegaskan pelajaran utama exercise ini: kernel saja $19\times$ lebih cepat dari CPU pada $n = 10^7$, tetapi sekali transfer PCIe ikut dihitung, operasi penuh justru $\approx 3\times$ lebih lambat dari CPU (speedup 0.34). GPU menang besar hanya bila data residen di device dan dipakai ulang.

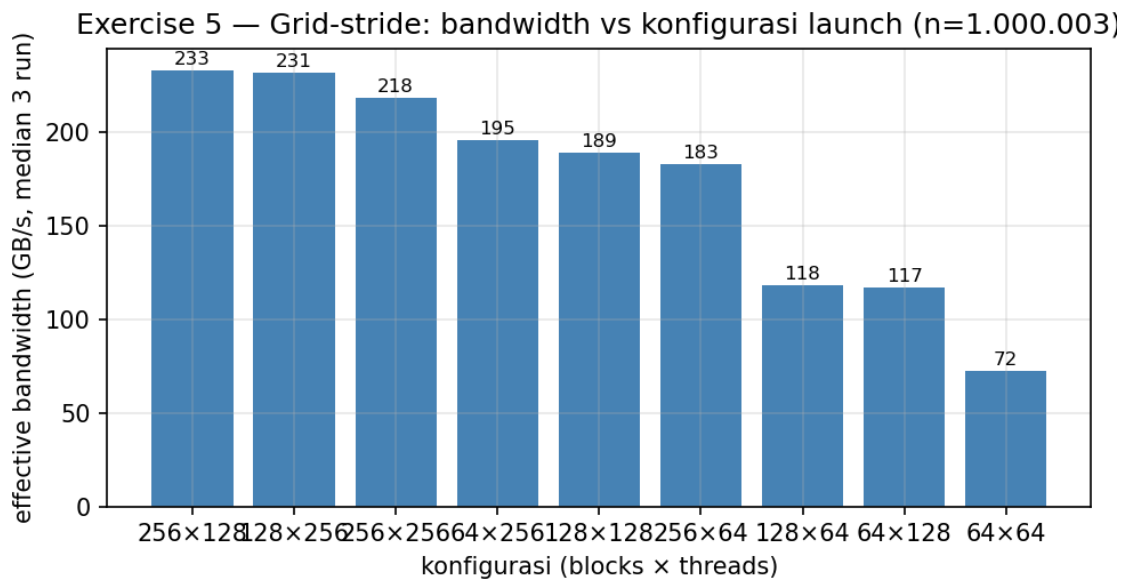
Exercise 5: Kernel grid-stride, tuning launch, bandwidth efektif

Berkas: `src/vector_gridstride.cu`. Kernel memakai pola *grid-stride loop*: tiap thread memproses indeks `tid`, `tid+stride`, `tid+2*stride`, ... dengan `stride` sama dengan jumlah thread total, sehingga konfigurasi block/thread bebas dipilih tanpa mengubah kebenaran — kernel tetap mencakup seluruh n lewat stride. Bandwidth efektif dihitung sebagai $3 \times n \times 4$ byte (baca `a`, `b`, tulis `c`) dibagi waktu kernel.

Sweep 9 konfigurasi (64/128/256 block $\times$ 64/128/256 thread, 3 run tiap konfigurasi, median pada Tabel 4 dan Gambar 6):

Tabel 4: Exercise 5 — median 3 run per konfigurasi, $n = 1000003$.

block	thread	kernel (ms)	bandwidth efektif (GB/s)
256	256	0.0530	226.45
128	256	0.0534	224.69
256	128	0.0547	219.43
64	256	0.0628	191.13
128	128	0.0638	187.97
256	64	0.0692	173.45
128	64	0.0986	121.71
64	128	0.1006	119.28
64	64	0.1677	71.55

Gambar 2: Bandwidth efektif terhadap konfigurasi launch (median 3 run, $n = 1000003$). Konfigurasi 256 block $\times$ 256 thread tercepat (226 GB/s); 64 $\times$ 64 paling lambat (72 GB/s).

Konfigurasi terbaik (256 $\times$ 256, 226 GB/s) $\approx 3.2\times$ lebih cepat dari yang terburuk (64 $\times$ 64, 72 GB/s). Polanya jelas: thread block yang lebih besar (256) konsisten menang karena occupancy dan coalescing akses memori yang lebih baik; 65536 thread total (256 $\times$ 256) memberi cukup paralelisme untuk menjenuhkan 40 SM pada T4.

Exercise 6: Reduksi dengan shared memory

Berkas: `src/reduce_sum.cu`. Tiap block mereduksi 256 elemen memakai array *shared memory* 256 elemen, pola halving dengan `__syncthreads()` tiap tahap, lalu menjumlahkan hasil per-block di host. Untuk `n=1000003`: `expected=3000003 result=3000003 match=yes`. Uji ukuran tepi semuanya cocok:

n	expected	result	match
1	0	0	yes
255	759	759	yes
256	762	762	yes
257	766	766	yes
1000003	3000003	3000003	yes

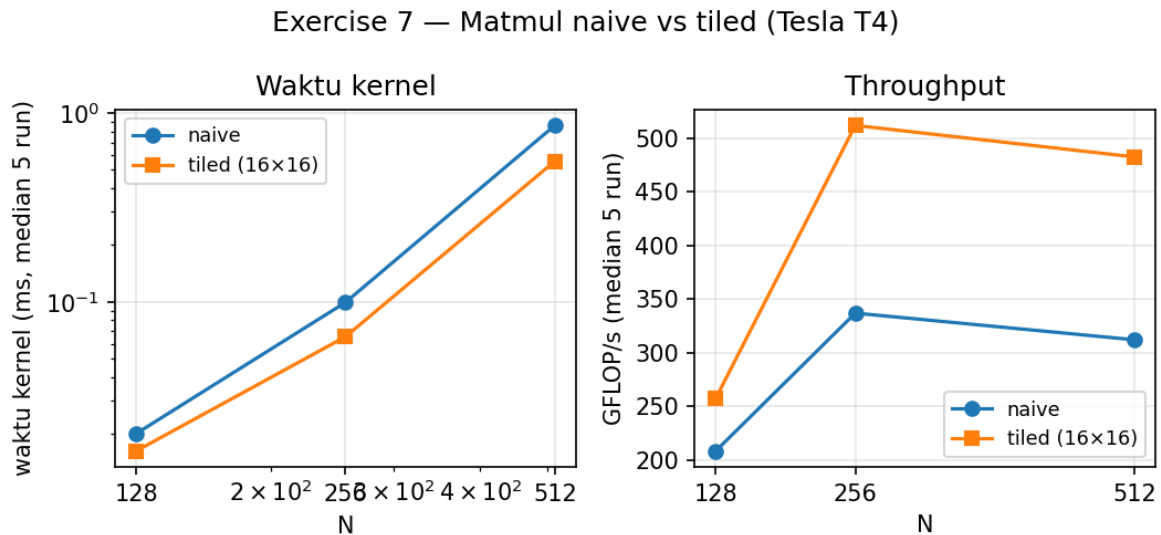
Exercise 7: Perkalian matriks naive vs tiled

Berkas: `src/matrix_mul.cu` (naive) dan `src/matrix_mul_tiled.cu` (tiled 16×16). Benchmark `src/matrix_bench.cu` menguji keduanya; varian tiled dipilih via `-DUSE_TILED`. Kebenaran dicek pada matriks non-persegi $127 \times 130 \times 129$: kedua varian `errors=0`, `max_error=0.000e+00`.

Perbandingan waktu pada ukuran persegi (5 run per varian per ukuran, median pada Tabel 5, Gambar 8):

Tabel 5: Exercise 7 — median 5 run per varian per ukuran (Tesla T4).

varian	N	kernel (ms)	GFLOP/s
naive	128	0.0201	208.38
tiled	128	0.0163	257.51
naive	256	0.0996	336.95
tiled	256	0.0655	512.00
naive	512	0.8602	312.08
tiled	512	0.5562	482.60



Gambar 3: Waktu kernel (kiri, skala log) dan throughput (kanan) matmul naive vs tiled terhadap ukuran matriks (median 5 run).

Varian tiled konsisten $\approx 1.5\times$ lebih cepat ($N=512$: 0.556 ms vs 0.860 ms; 482.6 vs 312.1 GFLOP/s) karena ubin 16×16 di shared memory memotong akses berulang ke memori global sebesar faktor 16. Seluruh 30 run benchmark `errors=0`.

Exercise 8: Diagnosis error dengan Compute Sanitizer

`compute-sanitizer --tool memcheck` tersedia pada CUDA 12.8 dan dijalankan pada `vector_add`, `reduce_sum`, dan `matrix_mul_tiled`: ketiganya **ERROR SUMMARY: 0 errors** — tidak ada out-of-bounds atau akses ilegal pada program yang benar.

Dua bug terkontrol kemudian disuntikkan pada *salinan* berkas (aslinya tidak disentuh):

1. **Tulis di luar batas** pada salinan `vector_add` (indeks ditulis sejauh n elemen melewati akhir buffer): sanitizer melaporkan **CUDA error: unspecified launch failure** dengan 97 error. Kernel yang menulis ke memori ilegal menggagalkan launch secara keseluruhan.
2. **Menghapus barrier pertama** pada salinan `reduce_sum`: `--tool racecheck` melaporkan hazard baca/tulis di baris 16 dan hasil reduksi salah (`result=2644645`, `expected=3000003`, `match=no`). Tanpa barrier, thread membaca `sdata` yang belum selesai ditulis thread lain — race condition klasik yang hanya muncul pada eksekusi paralel.

Mini-project: Stencil 2D lima titik

Berkas: `src/stencil.cu`. Kernel menerapkan stencil lima titik (tengah + atas/bawah/kiri/kanan) pada grid 2D; guard menangani tepi domain. Dua mode waktu diukur: *resident* (satu H2D, k langkah kernel, satu D2H) dan *per-step* (transfer bolak-balik tiap langkah). Hasil pada Tabel 6, seluruhnya **errors=0** (verifikasi terhadap CPU).

Tabel 6: Mini-project stencil — median waktu (Tesla T4).

kasus	CPU	kernel residu	operasi residu	per-step transfer
257×259 , 10 langkah	1.559 ms	0.061 ms	0.238 ms	1.562 ms
1024×1024 , 20 langkah	50.648 ms	0.936 ms	3.103 ms	36.014 ms

Pola Exercise 4 terulang dengan tajam: kernel residu $54\times$ lebih cepat dari CPU pada 1024^2 , tetapi memindahkan data bolak-balik tiap langkah (36.0 ms) $\approx 12\times$ lebih lambat dari strategi residu (3.1 ms). Untuk stencil iteratif, data *harus* tinggal di device antar langkah.

Refleksi: perlambatan, perbaikan, dan bug

Perlambatan — transfer mendominasi. Pada Exercise 4, speedup termasuk transfer tidak pernah melewati 1 (maksimum 0.34 pada $n = 10^7$): operasi penuh justru $\approx 3\times$ lebih lambat dari CPU meski kernelnya $19\times$ lebih cepat. Stencil mengonfirmasi dari sisi lain: transfer per-langkah $12\times$ lebih lambat dari eksekusi residu. Pelajarannya: bandwidth PCIe (≈ 25 ms untuk 120 MB pada $n = 10^7$) adalah biaya tetap yang hanya terbayar bila komputasi cukup besar atau data dipakai ulang.

Perbaikan — tiling dan tuning launch. Tiled matmul mencapai 482.6 GFLOP/s vs 312.1 GFLOP/s naive pada $N=512$ ($1.55\times$) lewat pemakaian ulang data di shared memory. Pada grid-stride, konfigurasi 256×256 (226 GB/s) mengalahkan 64×64 (72 GB/s) sebesar $3.2\times$ — thread block besar memberi occupancy dan coalescing yang lebih baik pada 40 SM T4.

Bug yang didiagnosis — ekspansi variabel IPython. Pada run pertama, lima cell loop shell (`!for n in ...; do ... $n ...; done`) semuanya berjalan dengan $n = 10000003$: Jupyter mengintersep `$n` dan menggantinya dengan variabel *Python* `n` (bernilai 10000003 dari cell ringkasan),

bukan variabel loop shell. Gejalanya jelas pada output: lima baris identik `n=10000003` dan, pada cell benchmark matriks, peringatan *integer overflow* diikuti **CUDA error: invalid configuration argument** karena ukuran matriks menjadi 10000003×10000003 . Diagnosis: `!` menjalankan perintah lewat IPython yang melakukan ekspansi `$`; perbaikannya mengganti `!` dengan magic `%%bash` sehingga `$n` menjadi milik bash sepenuhnya. Setelah perbaikan, seluruh ukuran/konfigurasi berjalan benar (20 run Exercise 4, 27 run Exercise 5, 30 run Exercise 7).

Reproduksi

Seluruh pekerjaan direproduksi lewat notebook Kaggle pada repositori:

1. Unggah `cuda_tutorial_kaggle.ipynb` via **File** → **Import Notebook**.
2. Pada panel kanan, set **Accelerator** ke **GPU**.
3. **Run All**; unduh notebook yang sudah tereksekusi via **File** → **Download as .ipynb**.

Struktur repositori: `src/` (13 berkas `.cu/` `.cuh` untuk semua exercise dan mini-project), `evidence/` (output notebook + CSV data mentah 77 pengukuran), `latex_report/` (laporan ini + gambar), dan notebook Kaggle asli. Flag kompilasi seragam: `-std=c++17 -O3 -lineinfo -arch=native`. Jika `-arch=native` gagal pada GPU lain, ganti dengan `-arch=sm_70` (P100) atau `-arch=sm_75` (T4).

Kesimpulan

Kedelapan exercise dan mini-project stencil diselesaikan pada Tesla T4 (CC 7.5, CUDA 12.8) dengan seluruh uji kebenaran lolos, termasuk ukuran tepi. Pengukuran menunjukkan tiga pelajaran utama: (1) kernel GPU mencapai speedup $19\times$ atas CPU untuk data residu, tetapi transfer PCIe membuat operasi end-to-end lebih lambat dari CPU bila data hanya dipakai sekali; (2) tiling shared memory memberi $1.55\times$ pada matmul dan tuning launch memberi hingga $3.2\times$ pada bandwidth; (3) Compute Sanitizer bersih pada program benar dan menangkap kedua bug terkontrol (out-of-bounds write dan race condition reduksi). Satu bug alur kerja nyata (ekspansi `$n` oleh IPython) didiagnosis dan diperbaiki dengan `%%bash`; seluruh angka pada laporan ini berasal dari run setelah perbaikan.